
SRD Arena

Projektbericht

Arbeitsfassung

Einführung in Python

Heinrich-Heine-Universität Düsseldorf

31. August 2026

Inhaltsverzeichnis

1	Einleitung	1
1.1	Motivation	1
1.2	Zielsetzung und Umfang	1
2	Bibliotheken und Entwicklungswerkzeuge	2
3	Konzept und Implementierung	3
3.1	Programmstruktur	3
3.2	Datenimport und Validierung (content)	4
3.3	Fachmodell und Regellogik (domain)	4
3.4	Anwendungssteuerung und Schnittstellen (engine)	4
3.5	Technischer Ablauf am Beispiel eines Zaubers	4
3.6	KI-Unterstützung im Entwicklungsprozess	5
4	Ergebnisse	8
4.1	Funktionsnachweis	8
4.2	Beispieldurchlauf eines Encounters	8
5	Diskussion und Fazit	9
5.1	Zielerreichung	9
5.2	Herausforderungen und Abweichungen vom Plan	9
5.3	Kritische Würdigung	10
5.4	Ausblick	10
6	Literatur- und Quellenverzeichnis	10
7	Anhang	10
7.1	Bedienungsanleitung	11
7.2	Ausgewählte Code-Snippets	11

1 Einleitung

Dungeons and Dragons (D&D) ist ein beliebtes Pen and Paper-Rollenspiel, in dem taktische, rundenbasierte Kämpfe (Encounter) einen großen Teil des Spiels ausmachen. Üblicherweise kämpft ein Team von Spielern gegen ein Team von NPCs (Non-Player-Characters), die vom Spielleiter gesteuert werden. Das System Reference Document (SRD) 5.2.1 ist eine unter der Creative Commons License CC-BY-4.0 verfügbare Fassung der Spielregeln, die den größten Teil der Grundmechaniken des Spiels enthält, in der aber viele Inhalte des eigentlichen Spiels fehlen. So kann ein Spieler beispielsweise für seinen Charakter eine Klasse und Subklasse wählen, jedoch steht pro Klasse nur eine Subklasse zur Verfügung, während im proprietär lizenzierten Spielerhandbuch pro Klasse vier Subklassen zur Auswahl stehen. Dieses Programm, SRD Arena, setzt zunächst einen Teil des Kampfsystems des SRD 5.2.1 als rundenbasiertes 2D-Spiel um. Es gibt mehrere vordefinierte Kampfszenarien, die die umgesetzten Funktionalitäten demonstrieren. Anhand von Vorlagen können in weiteren Dateien auch weitere Szenarien vom Nutzer definiert und anschließend gespielt werden.

1.1 Motivation

Die meisten Spielleiter entwerfen die Kampfszenarien für ihre Spieler bereits vor dem eigentlichen Spieltermin. Aus Sicht des Spielleiters ist das Ziel eines Kampfes, den Spielern neben einem interessanten Narrativ auch eine taktische Herausforderung zu bieten, ohne dass der Kampf zu schwer wird. Im Rahmen der umgesetzten Regeln können Spielleiter mit diesem Programm ihre Kampfszenarien vor dem eigentlichen Spiel ausprobieren. Das ist bereits vorteilhaft, weil im Programm viele Berechnungen und Würfe von Würfeln automatisch stattfinden, die sonst händisch geschehen müssten. Mittelfristig, aber nach Abgabe des Projektes, soll der gesamte Umfang der SRD-Regeln abgedeckt werden.

Langfristig wäre es auch möglich, das Projekt um eine Option zu erweitern, die Kampfentscheidungen aller Teilnehmer durch Bots automatisiert. Dann könnten Kämpfe vielfach simuliert werden, was Daten erzeugen würde, die für den Spielleiter in noch robustere Kennzahlen über die Ausgewogenheit eines Kampfszenarios umgewandelt werden könnten. Diese Funktionalität ist geplant und stellt auch die eigentliche Motivation für das Projekt dar, kann aber aufgrund der beschränkten Bearbeitungszeit erst nach Abgabe des Projektes umgesetzt werden.

Durch eine große Zahl automatisch gespielter Kämpfe könnten einzelne Würfelergebnisse ausgeglichen werden. Für einen Spielleiter wäre dann nicht nur erkennbar, ob ein Encounter einmal gewonnen wurde, sondern auch, wie häufig ein Team gewinnt und welche Teilnehmer besonders großen Einfluss auf das Ergebnis haben. Die im Projekt umgesetzte Kampfungine soll dafür später die technische Grundlage bilden.

1.2 Zielsetzung und Umfang

SRD Arena ist eine Engine für taktische Kämpfe auf einem 2D-Spielfeld nach dem SRD 5.2.1-Regelwerk. In einer GUI kann der Nutzer Kampfszenarien spielen. Die Steuerung der Teil-

nehmer erfolgt wahlweise durch Nutzerinput, oder durch sehr simple Bots. In strukturierten Textdateien können Kampfszenarien definiert werden. Ein Kampf besteht aus Teams, die Mitglieder beinhalten. Jedes Mitglied hat eine Startposition auf dem Spielfeld, sowie eine Referenz auf eine Definition ihrer Attribute und Fähigkeiten.

Teilnehmer

Während des Kampfes verwaltet die Engine für jeden Teilnehmer unter anderem Lebenspunkte, Position, Bedingungen und verbleibende Aktionsressourcen. Werte wie Rüstungsklasse, Bewegungsgeschwindigkeit, Angriffe und bekannte Zauber stammen aus der zugehörigen Kreaturendefinition.

Rundenablauf

Zu Beginn des Encounters bestimmt ein Initiativwurf die Reihenfolge, in der Teilnehmer ihre Züge ausführen. Nach dem Zug des letzten Teilnehmers beginnt eine neue Runde. Während seines Zuges verfügt ein Teilnehmer grundsätzlich über Bewegung, eine Aktion und eine Bonusaktion. Die Engine verwaltet diese Ressourcen und verhindert ihre mehrfache Verwendung. Bestimmte Ereignisse können eine Reaktion eines anderen Teilnehmers auslösen und den laufenden Zug vorübergehend unterbrechen. Eine implementierte Anwendung ist der Gelegenheitsangriff beim Verlassen der Reichweite eines Gegners. Eine verbrauchte Reaktion wird zu Beginn des nächsten eigenen Zuges wieder verfügbar.

Das Spielfeld

Die Engine modelliert das Spielfeld als Gitter aus quadratischen Feldern. Die Seite eines Feldes ist fünf Fuß lang.

Der Umfang der Abgabe ist bewusst kleiner als der Umfang des SRD. Unterstützt wird jeweils ein einzelner Encounter mit ausgewählten Kampfreden. Nicht Teil der Abgabe sind eine vollständige Abbildung aller Klassen und Zauber, eine allgemeine Ausrüstungsverwaltung sowie mehrere aufeinanderfolgende Encounters. Diese Begrenzung ermöglicht es, die unterstützten Regeln innerhalb eines vollständigen Kampfablaufs zu verbinden und zu testen.

2 Bibliotheken und Entwicklungswerkzeuge

Zur Laufzeit verwendet SRD Arena hauptsächlich Pydantic und PySide6. Pydantic bildet die Schemata der JSON-Inhalte ab und validiert eingelesene Daten, bevor daraus Objekte des Fachmodells entstehen. Dadurch müssen Struktur- und Typprüfungen nicht für jedes Eingabeformat manuell implementiert werden. PySide6 stellt die Desktop-GUI bereit und verarbeitet die Eingaben sowie die Darstellung des Kampfgeschehens.

Für automatisierte Tests kommt pytest zum Einsatz. Hypothesis ergänzt beispielbasierte Tests um generierte Eingaben, mit denen unter anderem die Grenzen der Lebenspunktlogik geprüft werden; pytest-cov misst die dabei erreichte Testabdeckung. mypy prüft die Typannotationen, Ruff übernimmt statische Prüfungen und die einheitliche Formatierung, während Interrogate die Abdeckung durch Dokumentation misst.

Sphinx, MyST-Parser und Furo werden zum Erzeugen der Projektdokumentation verwendet. Erdantic und json-schema-for-humans erzeugen zusätzliche Darstellungen der Inhaltsmodelle und ihrer JSON-Schemata. Die Python-Abhängigkeiten sind in der pyproject.toml erfasst und werden mit uv installiert. uv wird außerdem verwendet, um die Entwicklungswerkzeuge in einer einheitlichen Projektumgebung auszuführen.

Für die Auswahl der Werkzeuge war vor allem ihre konkrete Aufgabe im Projekt entscheidend. Pydantic und PySide6 werden von der Anwendung zur Laufzeit benötigt. Die übrigen Pakete unterstützen Entwicklung, Prüfung und Dokumentation, ohne Teil der eigentlichen Kampfflogik zu werden. Zusätzliche Bibliotheken wurden nicht allein für kleine Hilfsfunktionen eingeführt, wenn dieselbe Aufgabe mit der Python-Standardbibliothek verständlich gelöst werden konnte.

3 Konzept und Implementierung

3.1 Programmstruktur

Der Quellcode ist nach Verantwortungsbereichen in die Pakete content, domain, engine und frontends gegliedert. Der Content-Bereich liest die in JSON-Dateien definierten Spieldaten, validiert sie und übersetzt sie in Domain-Objekte. Die Domain enthält den veränderlichen Encounterzustand sowie die davon unabhängigen Kampffregeln. Die Engine stellt mit Session, Commands und Observations eine frontendneutrale Schnittstelle bereit und koordiniert die Interaktion mit der Domain. Die Frontends setzen Benutzereingaben in Engine-Befehle um und stellen die zurückgegebenen Beobachtungen dar. Die Abhängigkeiten verlaufen überwiegend in Richtung der Domain: Diese kennt weder Content, Engine noch Frontends. Die Engine verwendet die Domain, ist aber nicht für das Laden von Dateien zuständig. Die Frontends steuern einen laufenden Kampf über die Engine-Schnittstelle. Direkte Zugriffe auf den Content-Bereich dienen lediglich der Encounter-Auswahl und Darstellungskonfiguration; die GUI verwendet zusätzlich zustandslose Geometriefunktionen für visuelle Vorschauen. Architekturtests prüfen diese Paketgrenzen. Die folgenden Abschnitte erläutern Datenverarbeitung, Regellogik und Anwendungssteuerung getrennt, bevor Abschnitt 3.5 ihr Zusammenspiel am Beispiel des Spellcastings zeigt.

Die Aufteilung richtet sich damit nach Verantwortlichkeiten und nicht nach einzelnen Spielfunktionen. Ein Zauber durchläuft mehrere Bereiche, ohne dass das Einlesen seiner JSON-Datei, seine Regelwirkung und seine Darstellung in derselben Klasse behandelt werden. Abbildung [X] kann diese Abhängigkeiten als Übersicht zeigen. Pfeile sollten dabei nur in die Richtung der tatsächlich erlaubten Importe weisen.

3.2 Datenimport und Validierung (content)

Die auf 5e.tools veröffentlichten JSON-Daten zum SRD-Regelwerk dienten als Grundlage für die selbst gepflegten Inhaltsdateien des Projekts.

Die Inhaltsdateien werden nicht direkt von der Kampflogik ausgewertet. Der Content-Bereich liest die JSON-Daten zunächst ein und prüft sie mit Pydantic-Schemata. Bei einem Encounter werden unter anderem positive Spielfeldmaße, eindeutige Teilnehmerkennungen, gültige Teamzuordnungen und unterschiedliche Startpositionen verlangt. Anschließend übersetzen Builder die validierten Daten in Objekte der Domain. Fehlerhafte Inhalte werden dadurch an der Dateigrenze abgelehnt, während die übrigen Bereiche mit bereits geprüften Daten arbeiten können.

3.3 Fachmodell und Regellogik (domain)

Die Domain enthält sowohl die Definition eines Encounters als auch seinen veränderlichen Zustand während eines Kampfes. Beim Start werden die geladenen Kreaturvorgaben in einen `EncounterState` übernommen. Dort werden Initiative, Positionen, Lebenspunkte, Aktionsressourcen und laufende Effekte verwaltet. Die Regeln für Bewegung, Angriffe, Rettungswürfe, Schaden, Heilung und Bedingungen arbeiten auf diesem Zustand, kennen aber weder JSON-Dateien noch Elemente der GUI.

Regelverletzungen werden möglichst verhindert, bevor eine Aktion angeboten wird. Vor der Ausführung werden ihre Voraussetzungen erneut geprüft, weil sich der Zustand seit der Darstellung geändert haben kann. Eine erwartbare ungültige Auswahl führt zu einer fachlichen Ablehnung mit Meldung und Fehlercode und nicht zu einem Programmabbruch. Unerwartete Programmierfehler werden innerhalb der Domain dagegen nicht pauschal abgefangen, damit ihr Traceback bei der Fehlersuche erhalten bleibt.

3.4 Anwendungssteuerung und Schnittstellen (engine)

Die Engine stellt mit `Session` die Schnittstelle für ein laufendes Spiel bereit. Ein Frontend erhält über `Observations` eine unveränderliche Sicht auf den aktuellen Entscheidungszustand und sendet seine Auswahl als typisierten Command zurück. Die Engine prüft, ob der Command noch zur aktuellen Entscheidung gehört, und übergibt eine gültige Aktion an die Domain. Das Ergebnis enthält anschließend die neue `Observation` sowie Meldungen und Ereignisse für die Darstellung.

Dadurch muss die GUI den Encounterzustand nicht selbst verändern und keine Kampfregeln nachbilden. Automatisch gesteuerte Teilnehmer verwenden denselben Ablauf; die Session führt ihre Aktionen nur so lange aus, bis wieder eine Eingabe von außen erforderlich ist. Neben der GUI könnte deshalb später auch ein Simulationsprogramm dieselbe Engine-Schnittstelle verwenden.

3.5 Technischer Ablauf am Beispiel eines Zaubers

Beim Laden eines Encounters werden zunächst alle Zauberdateien eingelesen und durch das Pydantic-Modell `SpellSchema` validiert. Der Builder überführt die validierten Daten anschließend in Domänenobjekte vom Typ `Spell`. Dabei wird insbesondere die deklarative capability

in eine ausführbare Definition übersetzt. Beim Beispiel Fireball beschreibt sie Zielgebiet, Rettungswurf, Schaden und Skalierung. Über die in der Kreatur hinterlegten Zauberreferenzen werden diese Objekte ihrem Spellcasting und damit den bekannten Zaubern zugeordnet; die JSON-Datei selbst wird zur Laufzeit nicht direkt ausgewertet. Ist die Kreatur am Zug, erzeugt `available_spell_actions` aus ihren bekannten Zaubern mögliche `EncounterAction`-Objekte. Vor der Anzeige prüfen Eligibility Rules unter anderem, ob der Zauber bekannt und implementiert ist, genügend Aktions- und Zauberplatzressourcen verfügbar sind und gültige Ziele existieren. Nach der Auswahl werden gegebenenfalls noch Ziel, Wirkungsbereich oder Zaubergrad ergänzt. Bei Fireball bestimmt beispielsweise ein gewählter Punkt den kugelförmigen Wirkungsbereich. Die ausgewählte Aktion gelangt über den `EncounterOrchestrator` zur Zauberausführung. Dort werden die Voraussetzungen erneut geprüft, Ressourcen verbraucht und anschließend die deklarative Fähigkeit aufgelöst. Für Fireball werden die betroffenen Kreaturen bestimmt, Geschicklichkeitsrettungswürfe durchgeführt, 8d6 Feuerschaden beziehungsweise halber Schaden berechnet und eine mögliche Skalierung durch höhere Zauberplätze berücksichtigt. Abschließend werden Schaden und weitere Effekte auf den `EncounterState` angewendet und als Meldungen sowie Ereignisse für die Frontends bereitgestellt.

Das Capability-Schema zerlegt die gemeinsame Struktur vieler Zaubers in Zielauswahl, Auflösung, Effekte und Skalierung. Fireball und Cone of Cold gehören dabei zur selben Familie: Beide führen einen Rettungswurf aus und verursachen bei einem Erfolg halben Schaden. Sie unterscheiden sich hauptsächlich durch Ursprung und Form des Zielgebiets, Schadensart und Schadenswürfel. Hold Person verwendet denselben Ablauf für Zielauswahl und Rettungswurf, erzeugt bei einem Fehlschlag aber den Zustand Paralyzed und einen wiederholten Rettungswurf am Ende des Zuges. Auf diese Weise können unterschiedliche Zaubers aus wiederverwendbaren Bausteinen zusammengesetzt werden, ohne für jeden Zauber einen vollständigen eigenen Ausführungspfad zu schreiben.

3.6 KI-Unterstützung im Entwicklungsprozess

Einsatzumfang und Arbeitsteilung

In diesem Projekt wurde der überwiegende Teil der Programmlogik mit KI generiert. Viele Ideen zur Struktur wurden auch mit KI-Unterstützung auf Lücken geprüft und angepasst. Außerdem wurden Regelabschnitte, die in JSON-Dateien noch in Prosaform gegeben waren, mithilfe von KI in das entworfene JSON-Schema übersetzt.

Eine eindeutige Trennung nach vollständig menschlich und vollständig durch KI erstellten Dateien ist bei diesem Projekt kaum möglich. Meist gab ich ein fachliches Ziel oder ein erkanntes Problem vor, ließ Codex einen Änderungsvorschlag erstellen und prüfte anschließend Code, Tests und Auswirkungen auf die übrige Architektur. Meine eigene Arbeit lag damit besonders in der Auswahl des Umfangs, der Formulierung der Regeln, der Bewertung der Vorschläge und der Entscheidung, wann eine vorhandene Struktur nicht weitergeführt werden sollte.

KI-generierte Bestandteile

Vorgabe: Aufführen, welche Module, Funktionen oder Tests maßgeblich mit KI-Unterstützung, beispielsweise durch ChatGPT oder Copilot, erstellt wurden. Außerdem begründen, weshalb sich

der KI-Einsatz dafür eignete, etwa bei Boilerplate-Code, Standardalgorithmen oder regulären Ausdrücken.

- Übersetzung von verbleibender Regelprosa in JSON (Spells)
- Schrittweise Implementierung von Regeln - Hit Points, Angriffe, Spells, etc.

KI-Unterstützung wurde vor allem für wiederkehrende Implementierungsschritte genutzt. Dazu gehörten ähnliche Schemafelder und Builder, Tests nach bereits vorhandenen Mustern sowie die Übertragung ausgewählter Zauberregeln in das Capability-Schema. Auch bei der schrittweisen Ergänzung von Lebenspunkten, Angriffen und Zaubern erzeugte Codex große Teile des ersten Entwurfs. Das war besonders dann nützlich, wenn ein im Projekt bereits verwendetes Muster auf weitere Inhalte übertragen werden sollte.

Selbst programmierte Bestandteile

Vollständig selbst geschriebene größere Module gibt es nicht. Nicht an Codex delegiert werden konnten jedoch die fachliche Auswahl der umgesetzten Regeln und die abschließende Bewertung, ob eine Lösung zum Projekt passt. Dazu gehörten insbesondere das Streichen nicht mehr benötigter Features, die Entscheidung für einen einzelnen Encounter und die Kontrolle von Grenzfällen anhand des SRD-Regeltexts. Diese Arbeit führte häufig dazu, dass generierter Code anschließend umgebaut oder wieder entfernt wurde.

Einfluss auf Architektur- und Designentscheidungen

Vorgabe: Darstellen, ob die KI bereits bei der Planung und Architektur des Projekts konsultiert wurde. Falls dies der Fall war, erläutern, welche Vorschläge übernommen wurden und an welchen Stellen bewusst von den Empfehlungen der KI abgewichen wurde, weil die eigene Einschätzung besser zum Projekt passte.

- Lernen über Layered Architecture
 - Schemadesign für Laden von Content
 - Lösungsvorschläge waren meist nach Iteration gut, aber das Erkennen der Problematik lag oft bei mir
1. Früheres Refactoring: Bei Implementierung von Zaubern wurde FFahrplan in Zusammenarbeit mit KI aufgestellt, aber der Code wurde schnell sehr unübersichtlich. Auf Nachfrage schlug Codex vor, dem Fahrplan vor einem Refactoring noch relativ weit zu folgen, aber ich entschloss, das Refactoring früher zu beginnen, um auch früher einen Zustand zu erreichen, in dem ich den Code leichter nachvollziehen und selbst anpassen kann.
 2. SRD Arena hatte in der experimentellen Phase noch einen Spieler und einen Gegner. Daraus ist gewachsen, dass es eine primäre Kreatur auf dem Spielbrett gibt. Viele weitere Strukturen wurden dann parallel einmal für "den Spieler", dann für seine Verbündeten und seine Gegner geschrieben, obwohl die Logik eigentlich einheitlich sein sollte.
 3. Recherche für Regelparsing in Grenzfällen, Beispiel Calm Emotions und Immunitäten.

4. Conditions:

- Unconscious implies Incapacitated and causes the creature to fall Prone.
- When Unconscious ends, Incapacitated ends with it.
- Prone can remain independently. -> Domänenwissen verhindert Abstraktion, die KI vornehmen wollte

5. Funktionsweise von Spells aus tagged Prosa extrahieren vs. JSON enrichen:

- Weniger Programmcode zum Parsen von Authored content
- Teile der Definition werden nicht in Logik versteckt

6. Beginnen Dodge, Disengage, etc. in Domain oder in content?

Die Vorschläge der KI wurden nicht unverändert übernommen. Beispielsweise empfahl die KI zunächst, vor der Umstrukturierung des Content-Pakets weitere repräsentative Zauber zu implementieren. Diese Reihenfolge wurde bewusst verworfen, da dadurch zusätzliche Funktionalität auf einer bereits als unübersichtlich erkannten Struktur aufgebaut worden wäre. Ebenso wurde eine erste Implementierung generischer Kreaturentyp-Anforderungen nicht dauerhaft in dieser Form beibehalten, sondern später in das breitere Capability-Schema überführt. Zunächst wurden die Zauberdaten um explizite ausführbare Mechaniken erweitert, um die spätere Implementierung auf einer konsistenten Datenbasis aufzubauen. Bei der Modellierung von Zuständen wurden Vorschläge der KI außerdem anhand des Regeltexts überprüft und korrigiert, insbesondere hinsichtlich der Unterscheidung zwischen Immunität und Unterdrückung einer Bedingung.

Die KI war besonders hilfreich, um mögliche Aufteilungen und Abhängigkeiten sichtbar zu machen. Die Richtung einer Umstrukturierung musste trotzdem aus dem Gesamtzustand des Projekts abgeleitet werden. Ein Beispiel ist die Trennung von Content, Domain und Engine: Codex half beim Verschieben einzelner Verantwortlichkeiten, während ich darauf achten musste, dass keine neuen Rückabhängigkeiten entstehen. Ein weiteres Beispiel sind voneinander abhängige Conditions. Eine allgemein wirkende Abstraktion war dort nicht automatisch regelkonform, weil etwa Unconscious zwar Incapacitated verursacht, Prone nach dem Ende von Unconscious aber unabhängig bestehen bleiben kann.

Codegenerierung, Prüfung und Iteration

Vorgabe: Die Zusammenarbeit mit der KI beschreiben. Dabei insbesondere erläutern, ob generierter Code direkt übernommen oder in kritischen Iterationsschleifen geprüft und überarbeitet wurde. Dazu gehören auch manuelle Korrekturen von KI-Fehlern wie Halluzinationen oder veralteter Syntax.

Die Zusammenarbeit erfolgte überwiegend in kurzen Iterationen. Nach einer Beschreibung des gewünschten Verhaltens untersuchte Codex die betroffenen Dateien und erstellte einen Änderungsvorschlag. Danach wurden Tests, mypy und Ruff ausgeführt. Bei Fehlern oder einer unpassenden Struktur wurde nicht nur die konkrete Fehlermeldung behoben, sondern erneut geprüft, ob die gewählte Verantwortung

im richtigen Paket liegt. Größere Änderungen wurden deshalb häufig durch weitere Aufräumschritte und Architekturtests abgesichert.

Generierter Code wurde nicht allein deshalb übernommen, weil die Tests erfolgreich waren. Mehrfach waren Lösungen lokal funktionsfähig, führten aber neue Sonderfälle oder parallele Modelle für denselben Sachverhalt ein. Solche Probleme traten besonders bei älteren Strukturen für Szenen, Ausrüstung und Spielercharaktere auf. Die Iteration bestand dann darin, den noch benötigten Anwendungsfall enger zu formulieren, überflüssige Teile zu entfernen und die verbleibende Lösung erneut gegen Tests und Regeltext zu prüfen.

4 Ergebnisse

4.1 Funktionsnachweis

Die umgesetzte Funktionalität lässt sich anhand der mitgelieferten Showcase-Encounter nachvollziehen. Sie decken unter anderem normale Angriffe und Multiattacks, Bewegung und Gelegenheitsangriffe, unmittelbaren Zauberschaden, Zustände, anhaltende Zaubereffekte, Heilung mit einer auf mehrere Ziele verteilten Ressource sowie einzelne Klassenfähigkeiten ab. Abbildung [X] zeigt die GUI während eines Encounters; die angebotenen Schaltflächen entsprechen dabei den Aktionen, die die Engine im aktuellen Zustand als zulässig meldet.

Zusätzlich werden die Regeln und Paketgrenzen durch automatisierte Tests geprüft. Zum Zeitpunkt der Abgabe liefen [Anzahl] Tests einschließlich der Doctests ohne Fehler durch. `mypy --strict`, `ruff check` und `ruff format --check` meldeten [Ergebnis ergänzen]. Interrogate erreichte eine Dokumentationsabdeckung von [Wert ergänzen]. Diese Ausgaben sollten als kurze Tabelle oder als Ausschnitt aus der Konsole dargestellt werden, weil sie die formalen Qualitätskriterien direkt belegen.

4.2 Beispieldurchlauf eines Encounters

Nach dem Start mit `w run srđ-arena` zeigt das Programm zunächst die verfügbaren Encounter an. Für diesen Beispieldurchlauf wird „Immediate Damage Spells“ gewählt. Der Encounter enthält den Zauberwirker Spectrum Adept sowie mehrere gegnerische Ziele auf einem 18 mal 12 Felder großen Spielfeld. Beim Start erzeugt die Engine den veränderlichen Encounterzustand aus den geladenen Definitionen und würfelt die Initiative.

Sobald Spectrum Adept an der Reihe ist, zeigt die GUI seine möglichen Bewegungen und Aktionen. Der Nutzer wählt Fireball und anschließend einen Zielpunkt auf dem Spielfeld. Eine Vorschau markiert den kugelförmigen Wirkungsbereich. Nach der Bestätigung bestimmt die Engine die betroffenen Ziele, führt für jedes Ziel einen Geschicklichkeitsrettungswurf aus, zieht den ausgewürfelten oder halbierten Schaden ab und verbraucht den verwendeten Zauberplatz. Die Ergebnisse erscheinen sowohl an den Kreaturen als auch im Kampfprotokoll. Abbildung [X] zeigt die Wahl des Zielgebiets, Abbildung [Y] den Zustand nach der Auflösung.

Danach geht die Initiative zum nächsten Teilnehmer über. Automatisch gesteuerte Ziele führen ihre hinterlegte einfache Aktion aus, bis erneut eine Nutzereingabe erforderlich ist. Der Ablauf aus Entscheidung, Auflösung und aktualisierter Darstellung wiederholt sich, bis nur noch ein Team kampffähige Teilnehmer besitzt. Anschließend meldet die Engine das Ergebnis und bietet einen Neustart oder das Beenden der Anwendung an.

5 Diskussion und Fazit

5.1 Zielerreichung

Das für die Abgabe eingegrenzte Ziel wurde erreicht. Encounter können aus JSON-Dateien geladen, validiert, in der GUI ausgewählt und bis zu ihrem Ende gespielt werden. Die Engine verwaltet Initiative, Züge, Bewegung und Aktionsressourcen und unterstützt die im Projekt dokumentierten Angriffe, Rettungswürfe, Zauber, Bedingungen und Reaktionen. Externe Eingaben und einfache automatisch gesteuerte Teilnehmer verwenden dabei dieselbe Engine-Schnittstelle.

Nicht erreicht wurde das langfristige Ziel einer vollständigen Umsetzung des SRD. Viele Zauber enthalten weiterhin Mechaniken, für die noch kein ausführbarer Capability-Baustein vorhanden ist. Auch Spielercharaktere, Ausrüstung und Bots sind nur in dem Umfang umgesetzt, der von den mitgelieferten Beispielen benötigt wird. Diese Einschränkungen widersprechen nicht dem zuletzt festgelegten Abgabumfang, begrenzen aber die Encounter, die ohne weitere Programmierung definiert werden können.

5.2 Herausforderungen und Abweichungen vom Plan

Eine wesentliche Schwierigkeit entstand aus der Vorgeschichte des Projekts. Die erste Version war als Textabenteuer mit Szenen und Übergängen aufgebaut und unterschied an vielen Stellen zwischen einem primären Spieler, Verbündeten und Gegnern. Für die heutige Encounter-Engine waren diese Annahmen nicht mehr passend. Im Verlauf der Arbeit wurden deshalb Szenen, Folgen mehrerer Encounter sowie Teile der Ausrüstungs- und Klassenlogik entfernt oder enger begrenzt. Im Nachhinein wäre es vermutlich einfacher gewesen, von vorne zu beginnen. Das CYOA-Projekt beinhaltete nicht genug nützliche Vorarbeit, um zu die Aufräumarbeiten für die entfernten Strukturen zu rechtfertigen.

Auch der ursprünglich stärkere Schwerpunkt auf Spielercharakteren wurde verändert. Für die Abgabe erwiesen sich Monster-Statblocks und die allgemeinen Kampfregeln als bessere Grundlage, weil damit mehr unterschiedliche Encounter erstellt werden konnten, ohne zunächst eine vollständige Klassenprogression zu modellieren. Gleichzeitig erforderte die große Vielfalt der Zauber eine Entscheidung zwischen vielen einzelnen Sonderimplementierungen und einem gemeinsamen Capability-Schema. Der Aufbau dieses Schemas nahm mehr Zeit ein, verringerte danach aber die Menge an zauberspezifischem Programmcode.

5.3 Kritische Würdigung

Gut funktioniert hat die Trennung zwischen geladenen Inhaltsdaten, Regellogik und Anwendungssteuerung. Sie macht inzwischen erkennbar, an welcher Stelle eine neue Prüfung oder Regel ergänzt werden muss. Die automatisierten Tests waren besonders während größerer Refactorings nützlich, weil sie unbeabsichtigte Änderungen an bereits unterstützten Regeln sichtbar machten. Das Capability-Schema hat sich außerdem für Zauber mit ähnlicher Struktur bewährt, etwa für unterschiedliche Formen von Flächenschaden.

Weniger gut funktionierte die frühe Erweiterung des Funktionsumfangs, bevor die grundlegenden Paketgrenzen stabil waren. Durch KI konnten schnell neue Features entstehen, dadurch wuchs aber auch Code weiter, dessen Annahmen nicht mehr zum späteren Ziel passten. Bei einem neuen Projekt würde ich deshalb zuerst einen vollständigen, aber sehr kleinen Ablauf vom Einlesen einer Datei bis zur Darstellung in der GUI umsetzen. Erst danach würde ich weitere Regeln ergänzen. Architekturtests und eine ausdrücklich festgehaltene Liste nicht unterstützter Funktionen würde ich ebenfalls früher einführen.

5.4 Ausblick

Als nächste Erweiterung bietet sich eine gezielte Vergrößerung des Capability-Schemas an. Weitere wiederkehrende Zaubermechaniken könnten ergänzt werden, ohne bereits unterstützte Bausteine erneut zu implementieren. Spielercharaktere sollten dagegen erst nach einem eigenen Entwurf für Klassenprogression und Ausrüstung erweitert werden, weil die derzeitige kleine Lösung nur auf die mitgelieferten Fighter-Beispiele zugeschnitten ist.

Langfristig sollen bessere Bots vollständige Encounter automatisch spielen können. Viele Wiederholungen desselben Encounters könnten dann Siegquoten, verbleibende Lebenspunkte und den Einfluss einzelner Aktionen erfassen. Damit würde das Projekt wieder an seine ursprüngliche Motivation anschließen: Ein Spielleiter könnte einen vorbereiteten Kampf nicht nur einmal ausprobieren, sondern seine Schwierigkeit anhand mehrerer Simulationen einschätzen.

Außerdem könnte mithilfe der Engine ein KI-Modell trainiert werden, das in der Lage ist, einzelne oder sogar mehrere Charaktere effektiv zu spielen.

6 Literatur- und Quellenverzeichnis

- System Reference Document 5.2.1 inkl. CC-BY-4.0-Lizenz
- Dateien von 5e.tools

7 Anhang

Der Anhang enthält die kurzen Schritte zum Starten und Bedienen des Programms sowie ausgewählte Ausschnitte aus den Inhaltsdateien. Die Ausschnitte dienen als Beleg für das beschriebene Capability-Schema; die vollständigen Dateien und der übrige Quellcode sind über das Repository verfügbar.

7.1 Bedienungsanleitung

Vorausgesetzt werden Python 3.14 oder neuer und *uv*. Nach dem Klonen des Repositorys installiert *uv sync* die in der *pyproject.toml* festgelegten Abhängigkeiten. Mit *uv run srd-arena* wird die Anwendung gestartet. Im ersten Fenster wählt der Nutzer einen Encounter aus. Während des Kampfes zeigt die Seitenleiste die für den aktiven Teilnehmer verfügbaren Aktionen. Ziele oder Zielgebiete werden anschließend auf dem Spielfeld ausgewählt und bestätigt. Nach dem Ende des Encounters kann derselbe Kampf neu gestartet oder die Anwendung beendet werden.

7.2 Ausgewählte Code-Snippets

Die folgenden Ausschnitte vergleichen die Capabilities von Fireball und Cone of Cold. Beide Zauber verwenden einen Rettungswurf und verursachen bei einem erfolgreichen Wurf halben Schaden. Die Unterschiede werden ausschließlich durch die Daten für Ursprung, Geometrie, Attribut, Schadenswürfel und Schadensart beschrieben. Dadurch verwenden beide Definitionen denselben Builder und dieselbe Auflösungslogik.

```
FIREBALL:
...
"capability":
  "target":
    "type": "area",
    "origin": "point_in_range",
    "geometry":
      "shape": "sphere",
      "radius_feet": 20
    ,
    "resolution":
      "type": "saving_throw",
      "ability": "dex",
      "failure":
        "effects": [
          "type": "damage",
          "dice": "8d6",
          "damage_type": "fire"
        ]
    ,
    "success_damage": "half"
    ,
    "scaling": [
      "type": "slot_level",
      "above_level": 3,
      "per_level": [
        "type": "damage_dice",
        "amount": "1d6"
      ]
    ]
  ]
```

```
CONE OF COLD:
...
"capability":
  "target":
    "type": "area",
    "origin": "self",
    "geometry":
      "shape": "cone",
      "length_feet": 60
    ,
    "resolution":
      "type": "saving_throw",
      "ability": "con",
      "failure":
        "effects": [
          "type": "damage",
          "dice": "8d8",
          "damage_type": "cold"
        ]
      ,
      "success_damage": "half"
    ,
    "scaling": [
      "type": "slot_level",
      "above_level": 5,
      "per_level": [
        "type": "damage_dice",
        "amount": "1d8"
      ]
    ]
  ]
```